Lesson 02 Demo 03

Building a Reactive Agent Workflow with LangGraph and Tavily

Objective: To demonstrate how to build a reactive AI agent using the LangGraph framework integrated with LangChain's Tavily search tool

This demo guides learners through creating a react-style agent using create_react_agent(), connecting it to a real-time web search function, and invoking the agent to respond dynamically to user queries. The focus is on highlighting how LangGraph enables modular, tool-enhanced agent workflows for tasks that require external information retrieval and context-aware decision-making.

Scenario: A product lead at a fintech startup wants to monitor global developments in real-time fraud detection methods for UPI systems. The manual process of tracking cybersecurity advancements and regulatory changes is inefficient and prone to oversight. The goal is to automate this workflow using LangGraph, where an agent uses a search tool to gather the latest research and another agent validates and summarizes the findings for internal teams.

Tools required: Visual Studio Code

Prerequisites: Tavily key, Azure OpenAI API key

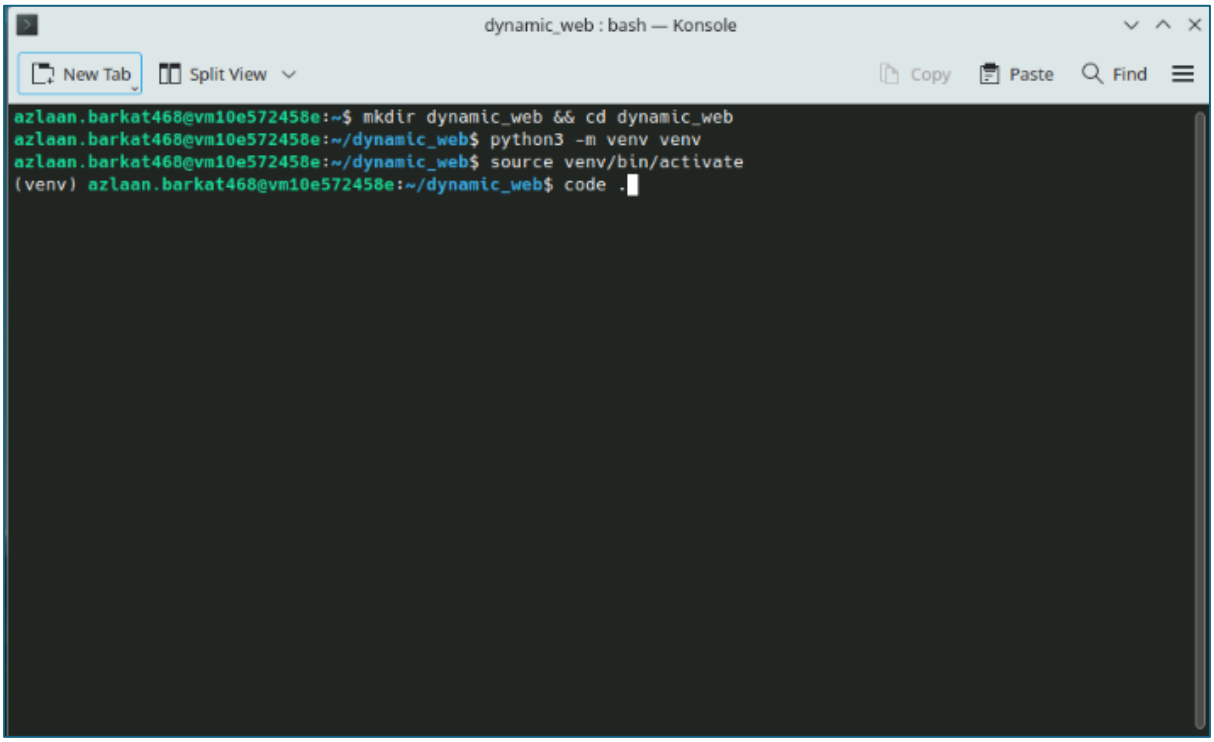
Steps to be followed:

1. Set up your local environment in Visual Studio Code
2. Configure the Azure OpenAI model and Tavily
3. Build the Tavily search tool
4. Create and run the reactive agent

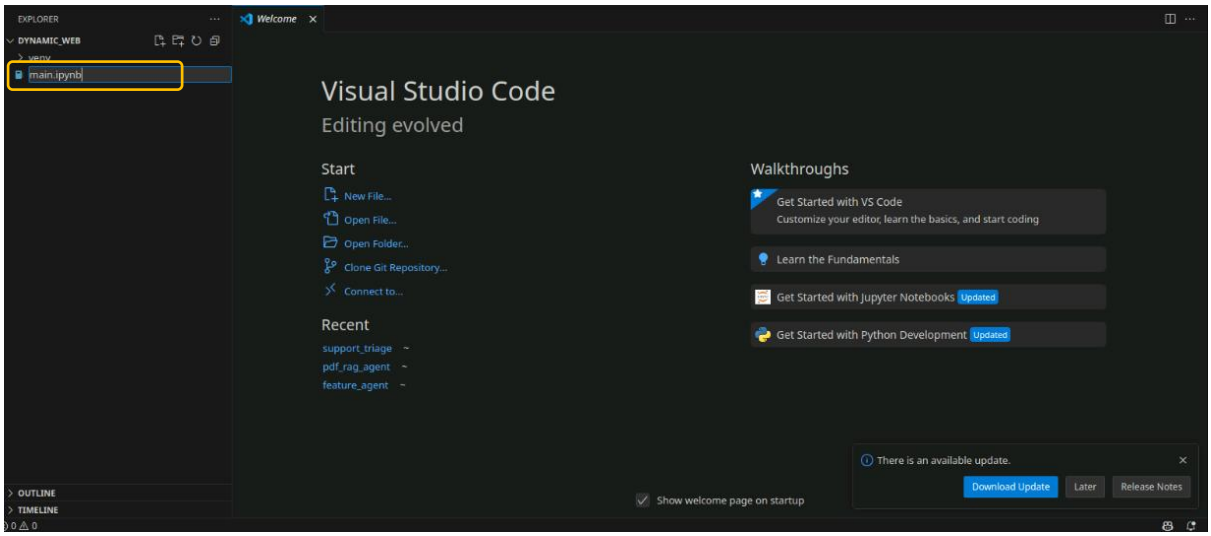
Step 1: Set up your local environment in Visual Studio Code

1.1 Open the terminal in your system and run the following commands:

mkdir dynamic_web && cd dynamic_web
python3 -m venv venv
source venv/bin/activate
code .



1.2 Create a new file: **main.ipynb**



1.3 In the first cell, write and run the following command to install required packages:

!pip install langgraph langchain-openai langchain-tavily tavily-python

```
# Step 1: Install required libraries
!pip install langgraph langchain-openai langchain-tavily tavily-python
```

```
[1] ✓ 6.5s                                                                Python
Collecting langgraph
  Downloading langgraph-0.6.6-py3-none-any.whl.metadata (6.8 kB)
Requirement already satisfied: langchain-openai in ./venv/lib/python3.12/site-packages (0.3.32)
Requirement already satisfied: langchain-tavily in ./venv/lib/python3.12/site-packages (0.2.11)
Requirement already satisfied: tavily-python in ./venv/lib/python3.12/site-packages (0.7.11)
Requirement already satisfied: langchain-core>=0.1 in ./venv/lib/python3.12/site-packages (from langgraph) (0.3.75)
Collecting langgraph-checkpoint<3.0.0,>=2.1.0 (from langgraph)
  Downloading langgraph_checkpoint-2.1.1-py3-none-any.whl.metadata (4.2 kB)
Collecting langgraph-prebuilt<0.7.0,>=0.6.0 (from langgraph)
  Downloading langgraph_prebuilt-0.6.4-py3-none-any.whl.metadata (4.5 kB)
Collecting langgraph-sdk<0.3.0,>=0.2.2 (from langgraph)
  Downloading langgraph_sdk-0.2.3-py3-none-any.whl.metadata (1.5 kB)
Requirement already satisfied: pydantic>=2.7.4 in ./venv/lib/python3.12/site-packages (from langgraph) (2.11.7)
Collecting xxhash>=3.5.0 (from langgraph)
  Downloading xxhash-3.5.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (12 kB)
Requirement already satisfied: openai<2.0.0,>=1.99.9 in ./venv/lib/python3.12/site-packages (from langchain-openai) (1.99.9)
Requirement already satisfied: tiktoken<1,>=0.7 in ./venv/lib/python3.12/site-packages (from langchain-openai) (0.11.0)
Requirement already satisfied: aiohttp<4.0.0,>=3.11.14 in ./venv/lib/python3.12/site-packages (from langchain-tavily) (3.12.15)
Requirement already satisfied: langchain<0.4.0,>=0.3.20 in ./venv/lib/python3.12/site-packages (from langchain-tavily) (0.3.27)
Requirement already satisfied: requests<3.0.0,>=2.32.3 in ./venv/lib/python3.12/site-packages (from langchain-tavily) (2.32.5)
Requirement already satisfied: httpx in ./venv/lib/python3.12/site-packages (from tavily-python) (0.28.1)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in ./venv/lib/python3.12/site-packages (from aiohttp<4.0.0,>=3.11.14->langchain-tavily)
Requirement already satisfied: aiosignal>=1.4.0 in ./venv/lib/python3.12/site-packages (from aiohttp<4.0.0,>=3.11.14->langchain-tavily) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in ./venv/lib/python3.12/site-packages (from aiohttp<4.0.0,>=3.11.14->langchain-tavily) (25.3.0)
                                                                Spaces: 4  {}  Cell 1 of 9
```

Step 2: Configure the Azure OpenAI model and Tavily

2.1 Use the OpenAI API key and the Tavily API key to build an LLM with Tavily search integration using the following code:

import os

 # ---- Azure APIM credentials ----
os.environ["OPENAI_API_KEY"] = "2ABecnfxzhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQJ99BDAC77bzfXJ3w3AAABACOGi3sC"
os.environ["OPENAI_API_BASE"] = "https://openai-api-management-gw.azure-api.net/openaiprodtest-clone/deployments/gpt-5-mini/chat/completions?api-version=2024-12-01-preview" # APIM base
os.environ["OPENAI_API_VERSION"] = "2024-12-01-preview"
os.environ["AZURE_DEPLOYMENT"] = "gpt-5-mini" # your deployment name

 # ---- Tavily Key ----
os.environ["TAVILY_API_KEY"] = "<Tavily key>" # Use your own Tavily key

```
import os

# ---- Azure APIM credentials ----
os.environ["OPENAI_API_KEY"] = "2ABecnfxzhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQJ99BDAC77bzfXJ3w3AAABACOGi3sC"
os.environ["OPENAI_API_BASE"] = "https://openai-api-management-gw.azure-api.net/openaiprodtest-clone/deployments/gpt-5-mini/chat/completions
os.environ["OPENAI_API_VERSION"] = "2024-12-01-preview"
os.environ["AZURE_DEPLOYMENT"] = "gpt-5-mini"   # your deployment name

# ---- Tavily Key ----
os.environ["TAVILY_API_KEY"] = "tvly-dev-*****************5" # Use your own Tavily key

[16] ✓ 0.0s                                                               Python
```

Note: You can create your own Tavily key by visiting the URL https://www.tavily.com/

2.2 Use **AzureChatOpenAI** from langchain-openai to create an LLM object:

```
# Create the LLM object (Azure OpenAI via langchain-openai)

from langchain_openai import AzureChatOpenAI

llm = AzureChatOpenAI(
    deployment_name=os.environ["AZURE_DEPLOYMENT"],   # gpt-5-mini
    api_key=os.environ["OPENAI_API_KEY"],
    azure_endpoint=os.environ["OPENAI_API_BASE"],     # APIM gateway endpoint
    api_version=os.environ["OPENAI_API_VERSION"],

)
```

Step 3: Build the Tavily search tool

3.1 Define a tool wrapper around Tavily API using the following code:

```
# Build Tavily Search Tool

from langchain_tavily import TavilySearch
from langchain.tools import tool

@tool
def search_tool(query: str):
    """Use this tool to search the web via Tavily API"""
    tavily_search_tool = TavilySearch(max_results=3)
    result = tavily_search_tool.invoke(query)
    return result
```

```python
# Build Tavily Search Tool

from langchain_tavily import TavilySearch
from langchain.tools import tool

@tool
def search_tool(query: str):
    """Use this tool to search the web via Tavily API"""
    tavily_search_tool = TavilySearch(max_results=3)
    result = tavily_search_tool.invoke(query)
    return result
```

[4] ✓ 0.6s Python

Step 4: Create and run the reactive agent

4.1 Create the agent with LangGraph and attach the tool using the following code:

 # Create a Reactive Agent with LangGraph and attach the Tavily tool
from langgraph.prebuilt import create_react_agent
agent = create_react_agent(model=llm, tools=[search_tool])

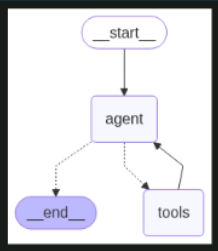
```python
# Create a Reactive Agent with LangGraph and attach the Tavily tool

from langgraph.prebuilt import create_react_agent

agent = create_react_agent(model=llm, tools=[search_tool])
```

[5] ✓ 0.0s Python

4.2 Run the following command to visualize the graph:
agent

```python
# Visualize the agent graph
agent
```

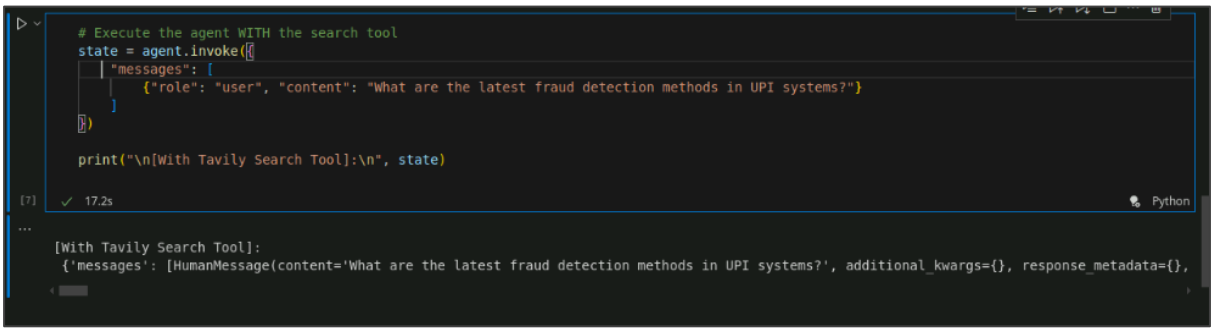
[6] ✓ 0.1s Python



4.3 Run the following code with the Tavily tool (real-time search):

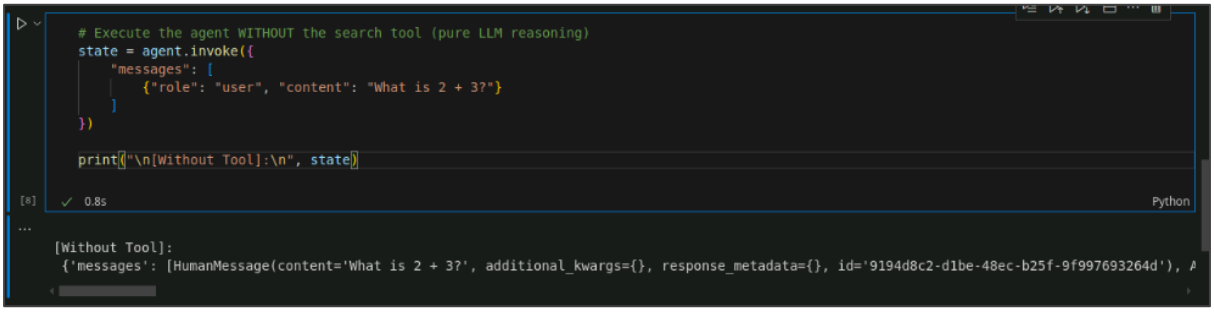
 # Execute the agent WITH the search tool
state = agent.invoke({

```
    "messages": [
        {"role": "user", "content": "What are the latest fraud detection methods in
UPI systems?"}
    ]
})

print("\n[With Tavily Search Tool]:\n", state)
```



4.4 Run the following code without the Tavily tool (pure LLM reasoning):

```
# Execute the agent WITHOUT the search tool (pure LLM reasoning)
state = agent.invoke({
    "messages": [
        {"role": "user", "content": "What is 2 + 3?"}
    ]
})

print("\n[Without Tool]:\n", state)
```



By following these steps, you have successfully built a reactive AI agent with LangGraph, integrated LangChain's Tavily search, created the agent using create_react_agent(), connected real-time web search, and invoked it to respond dynamically to user queries. This demonstrates LangGraph's modular, tool-enhanced workflows for external retrieval and context-aware decision-making.